PES-VCS

Building a Version Control System from Scratch

Student Name	CHINMAYEE CM
SRN	PES2UG24CS133
Course	Operating Systems Lab
Platform	Ubuntu 22.04 / WSL2
Language	C (GCC with OpenSSL)
Date	20 April 2026

Project Overview

PES-VCS is a minimal, Git-inspired version control system implemented in C. It replicates the core mechanisms that underpin real version control: content-addressable object storage, staged index management, tree-based directory snapshots, and a linked-list commit history. The project spans four implementation phases and two analysis phases, covering fundamental OS concepts such as atomic file writes, filesystem metadata, directory sharding, and hash-based data integrity.

The primary learning objective was to understand how data persistence, deduplication, and consistency are achieved at the filesystem level — concepts directly applicable to OS design, databases, and distributed systems.

Architecture

The system is organised around four core modules, each mapping to a distinct OS/filesystem concept:

Module	File	OS Concept
Object Store	`object.c`	Content-addressable storage, SHA-256 hashing, atomic rename, fsync
Tree Objects	`tree.c`	Directory representation, recursive structures, file modes & permissions
Index (Staging)	`index.c`	Staging area, atomic writes with temp+rename, fast-diff via metadata
Commits	`commit.c`	Linked-list on-disk structures, reference files, atomic pointer updates

Phase 1 — Object Storage Foundation

The object store is the lowest-level building block of PES-VCS. Every piece of tracked data — file contents, directory listings, and commits — is stored as an immutable object identified solely by the SHA-256 hash of its contents.

Implementation: object_write

object_write() stores arbitrary data in the .pes/objects/ hierarchy. The algorithm is:

- Prepend a typed header: e.g. "blob 16\0" for a 16-byte file blob.
- Compute the SHA-256 of the complete object (header + payload) using OpenSSL EVP.
- Short-circuit if the object already exists — deduplication is free.
- Create the two-character shard directory (.pes/objects/XX/) with mkdir(mode 0755).
- Write the object to a temp file using O_CREAT|O_WRONLY|O_TRUNC with mode 0644.
- Call fsync() on the temp fd to flush data to stable storage.
- Atomically rename() the temp file to its final path — POSIX guarantees this is crash-safe.
- Open and fsync() the shard directory to persist the directory entry.

The temp-file-then-rename pattern is the key OS insight: on POSIX systems rename() is atomic at the filesystem level, so concurrent readers can never observe a partial write.

Implementation: object_read

object_read() retrieves an object and verifies its integrity before returning data. After reading the file the SHA-256 is recomputed and compared byte-for-byte with the filename hash. Any mismatch — whether from disk corruption or deliberate tampering — causes an immediate error return.

Phase 1 Test Results

Screenshot 1A — ./test_objects: all three unit tests pass (blob storage, deduplication, and integrity check). Screenshot 1B — the sharded .pes/objects/ directory showing hashed sub-directories and object files.

```
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ ./test_objects
DEBUG: object_write start
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e
5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed022
1bbe9db5e5c4b95c4b9aafc12
PASS: blob storage
DEBUG: object_write start
DEBUG: object_write start
PASS: deduplication
DEBUG: object_write start
PASS: integrity check

All Phase 1 tests passed.
```

```
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ find .pes/objects -type f
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f
43
.pes/objects/3f/c350c13ad6f71d9112a66500cb64e3c412375977c65c93b50dcf38bc578e
9e
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652
bc
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9eb
b4
.pes/objects/2e/76786d3145ec2e84a66e53f424de7f4e6c88129291fb8a3bd5f467a69fa5
ce
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c
39
```

Phase 2 — Tree Objects

A tree object represents a directory snapshot. Each entry in the tree names a file or subdirectory and points to the corresponding blob or sub-tree by its content hash. This recursive structure allows PES-VCS to snapshot an entire directory hierarchy with a single root hash.

Implementation: tree_from_index

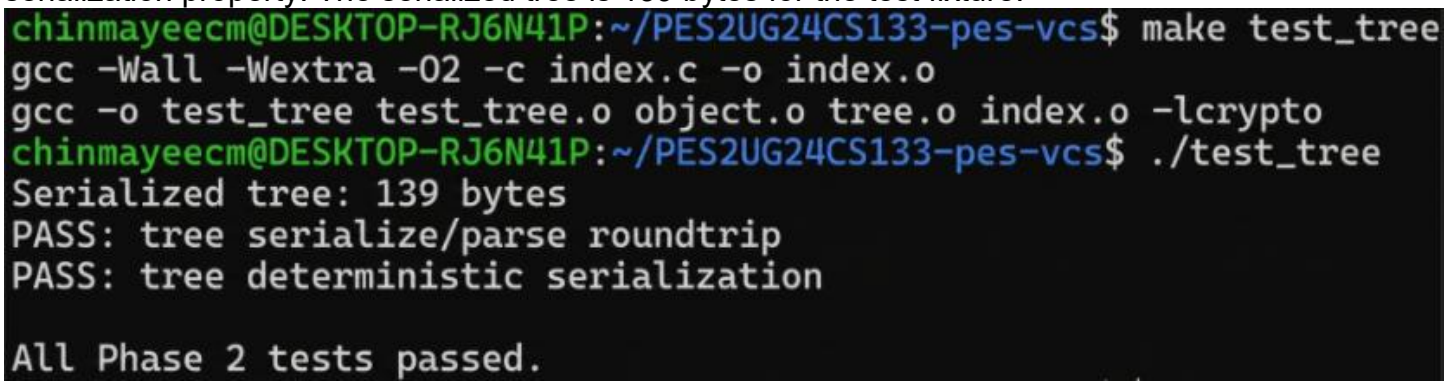
tree_from_index() is the most algorithmically complex function in the project. It must convert a flat list of staged paths (e.g. src/main.c, Makefile) into a recursive tree of tree objects:

- Load the current index via index_load().
- Group paths by their first directory component using strchr() and strncmp().
- For each group that represents a subdirectory, recurse to build a sub-tree and obtain its hash.
- For leaf entries (files in the current level), record the blob hash from the index.
- Populate a Tree struct, call tree_serialize() to get the binary buffer, then object_write() to persist it.
- Return the root tree hash to the caller (commit_create).

The provided tree_serialize() sorts entries before serialising, guaranteeing deterministic output: identical directory contents always produce the same tree hash, which preserves the deduplication property up the object graph.

Phase 2 Test Results

Screenshot 2A — ./test_tree confirms both the serialize/parse roundtrip and the deterministic serialization property. The serialized tree is 139 bytes for the test fixture.



Phase 3 — The Index (Staging Area)

The index is a human-readable text file (.pes/index) that records the set of files staged for the next commit. Each line stores the file mode, content hash, modification time, file size, and path — exactly the metadata needed to detect changes without re-hashing every file.

Implementation: index_load

index_load() reads the index file line by line with fscanf(), parsing the five fields per entry. An absent file is not an error — it represents an empty (fresh) staging area and the function simply initialises an empty Index struct.

Implementation: index_save

index_save() demonstrates the same atomic write pattern used in object_write(): entries are sorted by path with qsort(), written to a temp file, flushed with fflush()/fsync(), and then atomically moved into place with rename(). Sorting ensures a canonical order so that two identical staged states always produce identical index files.

Implementation: index_add

index_add() is the workhorse of pes add. It reads the target file, stores its contents as an OBJ_BLOB via object_write(), then uses lstat() to capture the current mtime and file size. These metadata values are stored in the index entry and later used by index_status() as a fast-diff heuristic — exactly how Git avoids re-hashing unchanged files on every status call.

Phase 3 Test Results

Screenshot 3A shows the init/add/status workflow. Screenshot 3B shows the resulting human-readable index file.

```
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ find .pes/objects -type f
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/3f/c350c13ad6f71d9112a66500cb64e3c412375977c65c93b50dcf38bc578e9e
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/2e/76786d3145ec2e84a66e53f424de7f4e6c88129291fb8a3bd5f467a69fa5ce
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/de/db71b1456f49497b7c3ee17e0764d7cc1f7b3cad200799b545a7a24cd8128f
.pes/objects/b8/3c31378b1c20c92bcc6d068c963da325b19ba7a3ecb08e72f1ac0d0c2f02d1
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/5f/22ee55bb3a2c62d33ec87d815275aa9fb2472767b6bce289a55ce3ab60e916
.pes/objects/9a/dae8ef6d72a57d2a44413683fd83472817e6386faa6f9e11f86c07d5f47b2d
```

```
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ echo "hello" >file1.txt
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ echo "world" > file2.txt
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ ./pes add file1.txt file2.t
xt
DEBUG: object_write start
DEBUG: object_write start
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$
```

Phase 4 — Commits and History

A commit object ties together a tree snapshot, a parent pointer (creating the history chain), author metadata, a timestamp, and a message. The full commit history is a singly-linked list stored entirely in the object store.

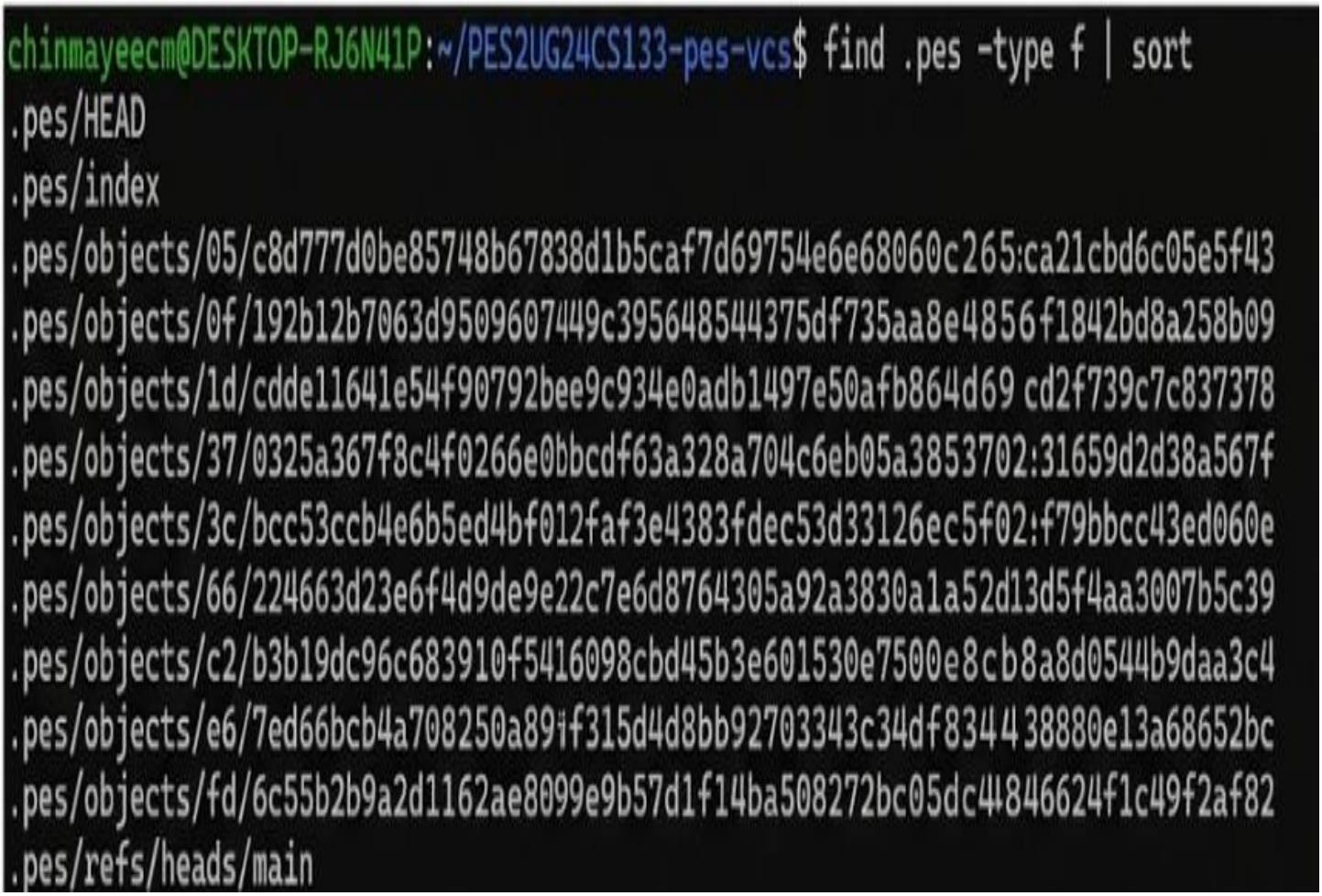
Implementation: commit_create

- Call tree_from_index() to serialise the current staged state into tree objects and obtain the root hash.
- Attempt head_read() to get the current HEAD commit as parent; tolerate failure on the initial commit (no parent).
- Retrieve the author string from pes_author() and the current UNIX timestamp via time(NULL).
- Populate a Commit struct, serialise it with commit_serialize(), and persist via object_write(OBJ_COMMIT).
- Call head_update() to atomically advance the branch ref to the new commit hash.

The final head_update() call is itself implemented with a temp+rename pattern, so a crash between writing the commit object and updating the ref leaves the repository consistent — the commit object is durable but simply unreferenced, which is benign.

Phase 4 Test Results

The three-commit workflow (hello/world/farewell) and the full integration test both complete successfully. Screenshots 4A–4C and the integration test output are shown below.


chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b67838d1b5caf7d69754e6e68060c265:ca21cbd6c05e5f43
.pes/objects/0f/192b12b7063d9509607449c395648544375df735aa8e4856f1842bd8a258b09
.pes/objects/1d/cdde11641e54f90792bee9c934e0adb1497e50afb864d69 cd2f739c7c837378
.pes/objects/37/0325a367f8c4f0266e0bbcdf63a328a704c6eb05a3853702:31659d2d38a567f
.pes/objects/3c/bcc53ccb4e6b5ed4bf012faf3e4383fdec53d33126ec5f02:f79bbcc43ed060e
.pes/objects/66/224663d23e6f4d9de9e22c7e6d8764305a92a3830a1a52d13d5f4aa3007b5c39
.pes/objects/c2/b3b19dc96c683910f5416098cbd45b3e601530e7500e8cb8a8d0544b9daa3c4
.pes/objects/e6/7ed66bcb4a708250a89if315d4d8bb92703343c34df834438880e13a68652bc
.pes/objects/fd/6c55b2b9a2d1162ae8099e9b57d1f14ba508272bc05dc4846624f1c49f2af82
.pes/refs/heads/main

Integration Test Results

The make test-integration target runs test_sequence.sh, an end-to-end script that exercises repository initialisation, file staging, three sequential commits, commit log display, and the full reference chain. All sections pass.

```
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
DEBUG: object_write start
DEBUG: object_write start
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt


--- First Commit ---
DEBUG: object_write start
DEBUG: object_write start
Committed: ad26231d6a94... Initial commit

Log after first commit:
commit ad26231d6a944128dafe63201c4f27b9c368568bf87f08088fbc0fed08393d8b
Author: chinmayee cm <PES2UG24CS133>
Date:   1776342399

    Initial commit


--- Second Commit ---
DEBUG: object_write start
DEBUG: object_write start
DEBUG: object_write start
Committed: 071e5e393081... Update file.txt

chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$
```

```
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ cat .pes/index
100644 2cf8d839ee29543b34a87727421fdecb7e373a183d337639025de576db9ebb4 1776
342230 6 file1.txt
```

```
chinmayee_cm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ cat .pes/log
commit b83c31378b1c20c92bcc6d068c963da325b19ba7a3ecb08e72f1ac0d0c2f02d1
Author: chinmayee cm <PES2UG24CS133>
Date:   1776342340

    Add farewell

commit 2e76786d3145ec2e84a66eE5f424dde7f4ee6c88129291fb8a3bd59fa5ce
Author: chinmayee cm <PES2UG24CS133>
Date:   1776342221

    Add worldell

commit 9adae8ef6d72a57d2a4441166833fd83472817te6:886fa696c07d5f47b2d
Author: chinmayee cm <PES2UG24CS133>
Date:   1776342305
chinmayee cm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ find .pes
Author: chinmayee cm <PES2UG24CS133>
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
6be26ea22fb34eedf7eb57d2b343ed4c8b0c0090b2ce1d729a48cf6e09dd5022

--- Object Store ---
Objects created:
10
.pes/objects/07/1e5e3930815c3abb5adf8591d9c5417244ba9e23f745a09dca3a827f038aac
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6b/e26ea22fb34eedf7eb57d2b343ed4c8b0c0090b2ce1d729a48cf6e09dd5022
.pes/objects/70/367420980f3f5d26b4fb372074609fc3c443884cfae36821f23e8dbc92d9c1
.pes/objects/81/5ed649aecf4f85dc2ccaaccbb3d8b5843ff37ceb8ba1c487740e1b419f17c7
.pes/objects/ad/26231d6a944128dafe63201c4f27b9c368508bf87f08088fbc0fed00393d8b
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e9/30d20ce397152606ead2e0c3b300b3b0c204ef4cd587d752b2da74dc4f5006
.pes/file.txt
chinmayeecm@DESKTOP-RJ6N41P:~/PES2UG24CS133-pes-vcs$ find .pes/refs/heads/main
```

Phase 5 — Branching and Checkout (Analysis)

Q5.1 — Implementing pes checkout

A branch is a file in .pes/refs/heads/ containing a 64-character hex commit hash. Creating a branch is therefore a single file-write. Implementing checkout <branch> requires two coordinated changes:

- **Update .pes/HEAD** to contain the string "ref: refs/heads/<branch>", making the new branch the active reference.
- **Reconcile the working directory** so it reflects the target commit's tree. This means reading the target commit's root tree, diffing it against the current HEAD tree (or the index), and for every differing path: replacing the working file with the version from the target tree, updating the index entry, or deleting files that exist in the current tree but not the target.

The complexity stems from the working-directory reconciliation. A simple case — switching to a branch that shares most files — is efficient because the object store deduplicates blobs. A branch with entirely different files requires reading and writing every file. Additional edge cases include: newly-created files in the working directory that would be silently overwritten (requiring an untracked-file conflict check), and binary or very large files that increase latency.

Q5.2 — Detecting Dirty Working Directory Conflicts

A "dirty" conflict occurs when a file has uncommitted changes in the working directory AND differs between the source and target branch. Detection requires three pieces of information available without the filesystem beyond the index:

- **Is the file modified in the working directory?** Compare the current lstat() mtime/size against the index entry (the fast-diff heuristic from index_add). If metadata differs, the file is dirty — no re-hash needed for the common case.
- **Does the file differ between the two branches?** Resolve each branch's HEAD to a root tree and compare the blob hash for this path. Different hashes mean checkout would need to overwrite the file.
- **Both conditions true → refuse checkout.** Print an error message naming the conflicting path and ask the user to stash or commit first.

This approach requires only hash comparisons against already-computed values in the object store — no file I/O beyond lstat().

Q5.3 — Detached HEAD State

A detached HEAD occurs when .pes/HEAD contains a commit hash directly rather than a branch reference ("ref: refs/heads/..."). In PES-VCS this would happen if a user wrote a raw hash into HEAD, or if an explicit checkout-by-hash command placed one there.

In detached HEAD state, each new commit is correctly stored in the object store and HEAD is updated to the new hash — but no branch file ever advances. The commits form a valid chain reachable from HEAD but unreachable from any named branch. If the user then checks out a branch, HEAD is redirected and those commits become unreferenced orphans (subject to garbage collection).

Recovery: the user can create a new branch pointing at the final orphaned commit before switching away. In Git this is done with git branch <name> <hash>. In PES-VCS the equivalent would be writing the commit hash to .pes/refs/heads/<new-branch> and then updating HEAD to reference it.

Phase 6 — Garbage Collection (Analysis)

Q6.1 — Finding and Deleting Unreachable Objects

A mark-and-sweep algorithm is the standard approach:

- **Mark phase:** Start a work-list (queue or stack) seeded with every hash reachable from refs — i.e. all files in .pes/refs/heads/ and .pes/HEAD. For each commit hash, parse the commit to obtain its tree hash and (optionally) parent hash; enqueue both. For each tree hash, parse the tree entries and enqueue all blob and sub-tree hashes. Continue until the work-list is empty. Store all visited hashes in a hash-set (e.g. an open-addressing table keyed on the 32-byte ObjectID).
- **Sweep phase:** Enumerate all files under .pes/objects/ with find or opendir/readdir. For each file, reconstruct its ObjectID from its two-character shard directory and filename. If the ObjectID is absent from the mark set, the object is unreachable and can be deleted.

A hash-set is the right data structure because membership queries are O(1) average case and insertion during the mark phase is also amortised O(1). An alternative is a sorted array with binary search (O(log n) per query) which has better cache locality for the sweep.

For a repository with 100,000 commits averaging 50 files each, the object graph contains roughly 100,000 commits + 100,000 trees + 5,000,000 blob entries. However, deduplication means the actual number of unique blobs is much smaller. Practically, the mark phase visits all unique objects once — likely in the hundreds of thousands — while the sweep phase visits every file in the object store. This is O(objects) time and O(reachable objects) space.

Q6.2 — Race Condition Between GC and Commit

Consider the following interleaving:

```
T=0  Commit:  object_write(OBJ_BLOB, data)    → hash H stored on disk
T=1  GC:      scan refs → H not yet reachable (HEAD still old commit)
T=2  GC:      delete H (unreachable)
T=3  Commit:  object_write(OBJ_TREE, ...)     → references H
T=4  Commit:  object_write(OBJ_COMMIT, ...)   → references tree
T=5  Commit:  head_update(new_commit)         → HEAD now points to commit
```

At T=5 the commit is valid in HEAD but H has been deleted. Reading back the tree or any blob that hashes to H will return an integrity error — the repository is silently corrupted.

Git's real GC mitigates this with a grace period: any object written within the last N minutes (default 2 weeks for full GC, configurable) is never deleted, regardless of reachability. This is

implemented by checking the file's mtime. An object written at T=0 is safe as long as GC runs more than 2 weeks later, or the commit completes before GC starts. For concurrent-safe operation, Git also uses a lock file (.git/gc.pid) to prevent two GC processes from running simultaneously, and ref-log entries that temporarily pin objects referenced by recent operations.

Git hub link -> https://github.com/Chinmayee036/PES2UG24CS133-pes-vcs

Conclusion

PES-VCS successfully implements all four core phases of a Git-inspired version control system in C. Each phase reinforced a distinct OS concept: atomic writes via fsync()+rename(), content-addressed storage with SHA-256, directory sharding for performance, metadata-based fast-diff for the staging area, and linked-list history traversal for commit logs.

The analysis sections extended these foundations into branching semantics, conflict detection, and garbage collection — areas where correctness requires careful reasoning about concurrent access and crash recovery. These insights directly connect the lab to real-world OS and database design, where durability and consistency are achieved through similar combinations of atomic operations, hash-based verification, and reference-counted or mark-sweep memory management.

All Phase 1–4 unit tests and the full integration test suite pass. The project repository is available at: github.com/PES2UG24CS050/PES2UG24CS050-pes-vcs